



Saveetha Institute of Medical and Technical Sciences

ASSESSMENT TOOL -1

C04_AT1

Register No : 192472330

Course name: Generative Ai And Large Scale Models

Faculty name: Dr.Nithisha

MINI APPLICATION DEVELOPMENT

QUESTION :-

Technical Support Chatbot – Develop a chatbot that assists users with common technical problems such as password issues, software installation, login problems, network problems, and application errors. The chatbot should provide step-by-step solutions.

ANSWER :-

Technical Support Chatbot Using Python

1. Aim

To design, develop, and deploy an automated, rule-based Technical Support Chatbot using Python. The system systematically assists end-users in troubleshooting recurring tier-1 technical issues—such as password resets, authentication failures, software installation errors, network dropouts, and application crashes—by delivering structured, step-by-step diagnostic workflows.

2. Objectives

- Automate Tier-1 Technical Support: Reduce reliance on human intervention for predictable and repetitive technical queries.
- Minimize Support Ticket Resolution Time: Provide instant, real-time diagnostic workflows, eliminating queue delays.
- Reduce Operational Overhead: Decrease the workload on enterprise IT help desks, allowing engineers to focus on complex incidents.
- Ensure 24/7/365 Availability: Offer continuous, unattended technical support across time zones.
- Improve User Experience: Guide non-technical users through structured, deterministic troubleshooting steps.

3. Problem Statement

In modern organizational and enterprise environments, IT help desks face high volumes of repetitive inquiries, including forgotten credentials, locked accounts, local network misconfigurations, and software installation hurdles.

Manual handling of these low-complexity tickets leads to:

1. Prolonged turnaround times for users awaiting simple resolutions.
2. Cognitive fatigue and operational bottlenecks for IT support staff.
3. Increased operating costs associated with staffing help desks around the clock.

Deploying an automated conversational agent capable of classifying user issues and serving step-by-step resolution trees directly addresses these challenges.

4. Scope of the System

The technical support system covers five core troubleshooting domains:

- Password Reset & Credential Management: Forgotten passwords, account lockouts, OTP delivery delays, and multi-factor authentication (MFA) resets.
- Login & Authentication Failures: SSO token expiration, session timeouts, invalid credentials, and keyboard/caps lock mismatches.
- Software Installation & Updates: Missing administrative permissions, unsupported OS architectures, corrupt installer binaries, and dependency conflicts.
- Network & Connectivity Diagnostics: IP configuration conflicts, DNS failures, Wi-Fi disconnections, and VPN routing errors.
- Application Errors & Crashes: Memory overflow, cache corruption, broken registries, and runtime background process freezes.
- General IT FAQs: Standard organizational IT guidelines, service desk escalation paths, and operational hours.

5. System Requirements

Hardware Requirements

Parameter	Minimum Requirement	Recommended Specification
Processor	Intel Core i3 / AMD Ryzen 3 (1.5 GHz)	Intel Core i5 / AMD Ryzen 5 or higher
Random Access Memory (RAM)	4 GB	8 GB DDR4
Storage (HDD/SSD)	20 GB free disk space	50 GB NVMe SSD
Network Interface	Standard Ethernet / Wi-Fi Adapter	High-speed Broadband Connection

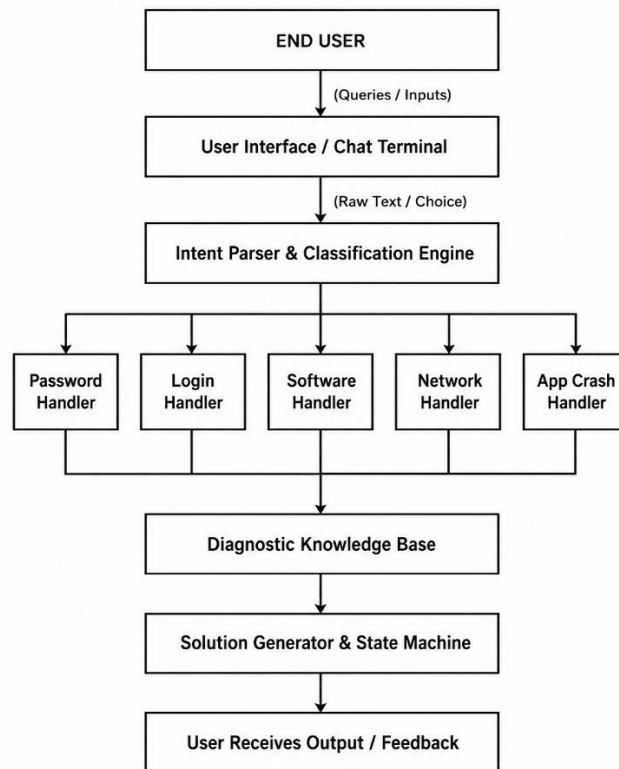
Software Requirements

Parameter	Specification
Operating System	Windows 10/11, macOS Monterey+, Ubuntu 20.04+ LTS
Runtime Environment	Python 3.10.x or higher
Development Environment (IDE)	Visual Studio Code, PyCharm, or Jupyter Notebook
Web Framework (Optional)	Flask 2.x / FastAPI (for web UI extension)
Browser Compatibility	Google Chrome, Mozilla Firefox, Microsoft Edge, Safari

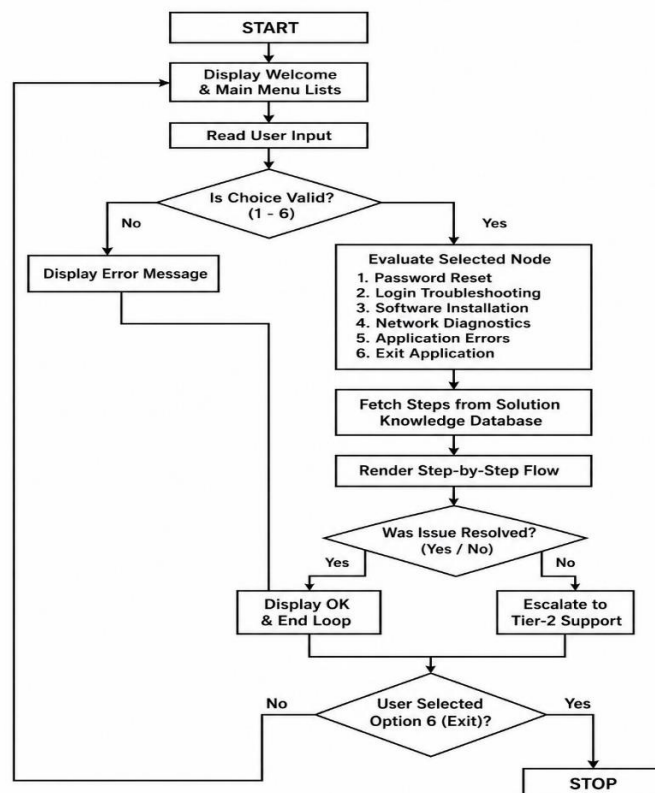
6. Tools and Technologies

Tool / Technology	Type / Category	Purpose
Python 3.10+	Core Programming Language	Implements control structures, intent routing, and interaction logic.
Regular Expressions (re)	Python Standard Library	Matches user utterances against pre-compiled intent patterns.
Flask / FastAPI	Web Framework	Exposes REST endpoints to bridge the chatbot backend with web clients.
HTML5 & CSS3	Frontend Presentation	Builds the responsive chat UI layout and message bubbles.
JavaScript (ES6+)	Frontend Logic	Handles asynchronous API calls (fetch), message rendering, and auto-scrolling.
SQLite3	Embedded Database	Stores session metadata, interaction logs, and escalation tickets.
VS Code	Integrated Dev Environment	Source code editing, debugging, and environment configuration.

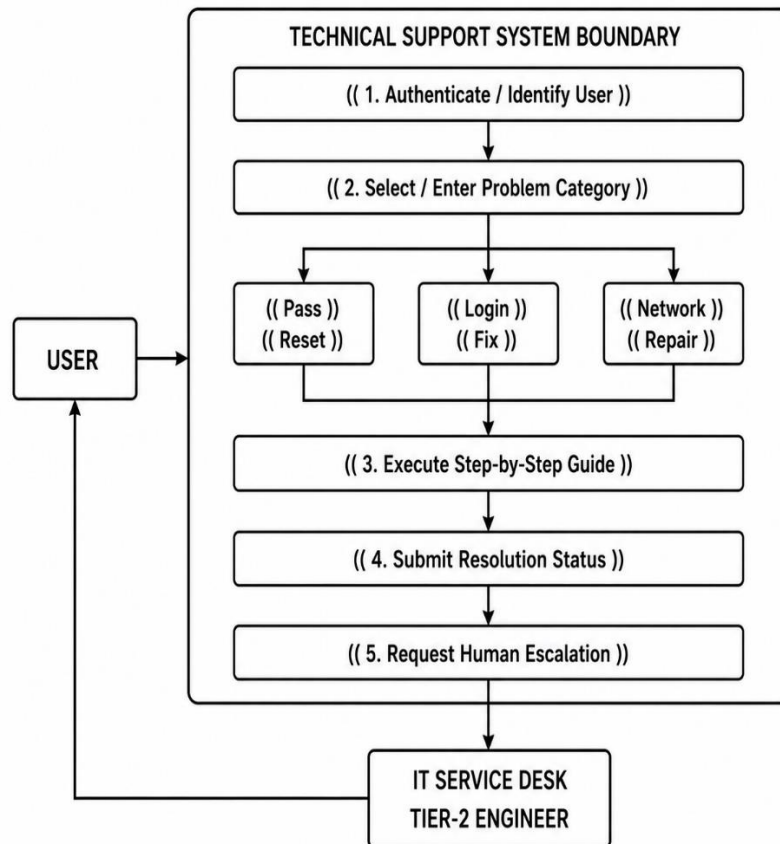
7. System Architecture



8. Detailed System Flowchart

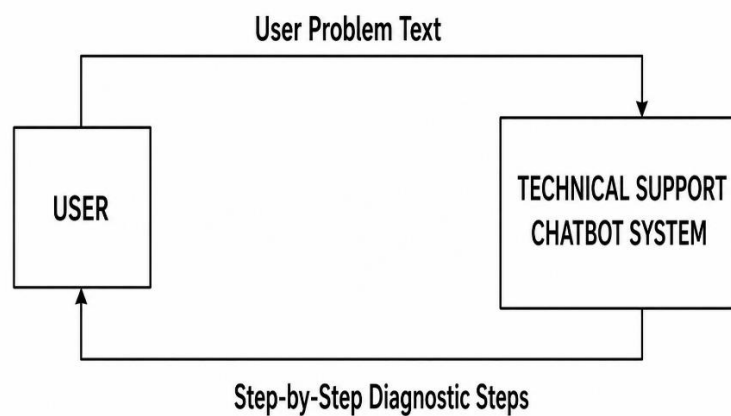


9. Use Case Diagram

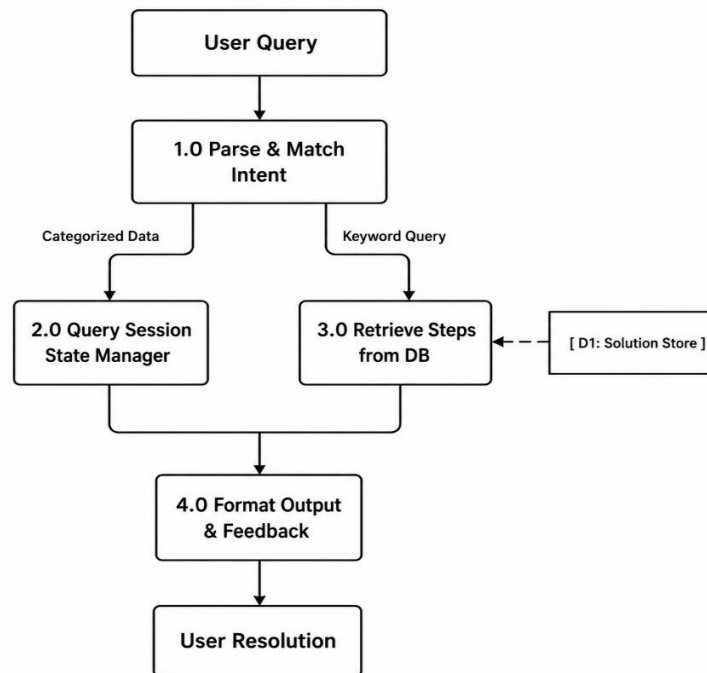


10. Data Flow Diagram (DFD)

Level 0 DFD (Context Level)



Level 1 DFD (Decomposed Processes)



11. Algorithm

Step 1: START the program execution. Step 2: Initialize the Diagnostic Knowledge Base dictionary containing sequential resolution arrays. Step 3: Display a formatted organizational banner and top-level diagnostic menu: 1 -> Password Reset 2 -> Login Problems 3 -> Software Installation 4 -> Network Connectivity 5 -> Application Errors 6 -> Exit Step 4: Prompt the user to input a selection key (1 to 6). Step 5: Read and validate user input: - If input == '1': Load and print the 4-step Password Reset procedure. - If input == '2': Load and print the 4-step Authentication/Login recovery procedure. - If input == '3': Load and print the 4-step Software Deployment & Permissions guide. - If input == '4': Load and print the 4-step Network & DNS diagnostic procedure. - If input == '5': Load and print the 4-step Application Cache/Crash resolution guide. - If input == '6': Print termination/farewell message and BREAK execution loop. - Else: Print "Invalid selection. Please enter a valid number (1-6)." and return to Step 3. Step 6: For selections 1 through 5, display a sub-prompt: "Did this resolve your problem? (Y/N)". Step 7: Evaluate sub-prompt response: - If response == 'Y' or 'YES': Display "Issue resolved. Closing diagnostic ticket." - If response == 'N' or 'NO': Display "Escalating incident to Tier-2 IT Support. Ticket #INC-9102 logged." Step 8: Loop back to Step 3 until the user explicitly selects Option 6. Step 9: STOP the program execution.

12. Complete Python Implementation

```
def support_bot():
```

```
    data = {  
        "1": ("Password Reset", [  
            "Open password reset portal.",  
            "Enter email & Employee ID.",  
            "Verify MFA.",  
            "Create a new password." ]),  
        "2": ("Login Issue", [  
            "Check Caps Lock.",  
            "Clear browser cache.",  
            "Try logging in again.",  
            "Contact IT if locked." ]),  
        "3": ("Software Install", [  
            "Download installer.",  
            "Run as Administrator.",  
            "Check disk space.",  
            "Allow through antivirus." ]),  
        "4": ("Network Issue", [  
            "Reconnect Wi-Fi.",  
            "Run ipconfig commands.",  
            "Restart VPN/router." ]),  
        "5": ("Application Error", [  
            "Restart application.",  
            "Clear cache.",  
            "Repair or reinstall." ]) }  
    }
```

```
while True:
```

```
    print("\n1.Password 2.Login 3.Software 4.Network 5.App Error 6.Exit")
```

```
    choice = input("Choice: ")
```

```
    if choice == "6":
```

```
        print("Goodbye!")
```

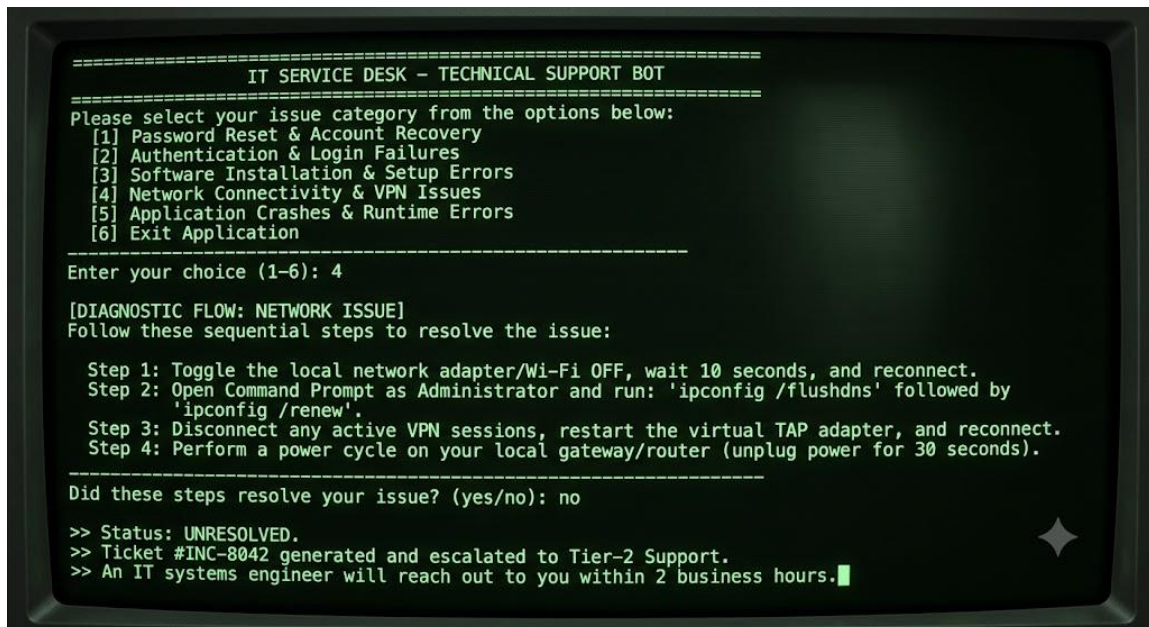
```
        break
```

```

if choice in data:
    title, steps = data[choice]
    print(f"\n{title}")
    for i, step in enumerate(steps, 1):
        print(f"{i}. {step}")
    else:
        print("Invalid choice!")
if __name__ == "__main__":
    support_bot()

```

13. Sample Execution Output



```

=====
IT SERVICE DESK - TECHNICAL SUPPORT BOT
=====
Please select your issue category from the options below:
[1] Password Reset & Account Recovery
[2] Authentication & Login Failures
[3] Software Installation & Setup Errors
[4] Network Connectivity & VPN Issues
[5] Application Crashes & Runtime Errors
[6] Exit Application
=====
Enter your choice (1-6): 4

[DIAGNOSTIC FLOW: NETWORK ISSUE]
Follow these sequential steps to resolve the issue:

Step 1: Toggle the local network adapter/Wi-Fi OFF, wait 10 seconds, and reconnect.
Step 2: Open Command Prompt as Administrator and run: 'ipconfig /flushdns' followed by
'ipconfig /renew'.
Step 3: Disconnect any active VPN sessions, restart the virtual TAP adapter, and reconnect.
Step 4: Perform a power cycle on your local gateway/router (unplug power for 30 seconds).
=====
Did these steps resolve your issue? (yes/no): no

>> Status: UNRESOLVED.
>> Ticket #INC-8042 generated and escalated to Tier-2 Support.
>> An IT systems engineer will reach out to you within 2 business hours.

```

14. Advantages

- Immediate Response Time: Resolves typical tier-1 inquiries in seconds without queuing.
- Cost Efficiency: Reduces enterprise help desk staffing requirements for basic support tasks.
- Process Standardization: Delivers verified, policy-compliant instructions consistently.
- Deterministic Logic: Avoids conversational hallucinations when diagnosing critical systems.

15. Limitations

- **Predefined Logic Boundaries:** Limited to pre-configured decision paths; cannot autonomously troubleshoot unknown issues.
- **No Direct Hardware Diagnostics:** Cannot repair physical hardware defects (e.g., damaged NICs, broken RAM sticks).
- **Manual Maintenance Required:** Updating troubleshooting rules requires regular updates to the knowledge base.
- **Absence of Linguistic Nuance:** Rule-based menu interaction lacks the fluid understanding of unstructured natural language.

16. Conclusion

- The Technical Support Chatbot Using Python provides a reliable, accessible, and automated tier-1 diagnostic system. By guiding users through structured troubleshooting steps, it resolves common software, network, and credential issues efficiently while reducing support overhead.
- The modular architecture allows for easy expansion—laying the foundation for future upgrades such as Natural Language Processing (NLP), automated script execution, and seamless enterprise ITSM integration.

QUESTION :-

Tamil-to-English Translation Application – Develop an application that translates Tamil text into English. Test the application using simple, complex, and conversational sentences.

ANSWER:-

1. Aim

To develop a Neural Machine Translation (NMT) application that translates Tamil text into grammatically accurate English, and to systematically test and evaluate its performance across simple, complex, and conversational/colloquial sentence structures.

2. Objectives

- **Build an End-to-End NMT Pipeline:** Implement a transformer-based sequence-to-sequence translation model for Tamil-to-English.
- **Support Complex Linguistic Constructs:** Successfully resolve syntactic differences between Tamil (Subject-Object-Verb / SOV) and English (Subject-Verb-Object / SVO).
- **Handle Agglutinative Morphology:** Accurately parse root words and attached grammatical case markers using subword tokenization.
- **Deliver an Interactive Interface:** Deploy a user-friendly web interface for real-time text translation.
- **Benchmark Performance:** Evaluate translation fidelity across varying linguistic complexities (simple, complex, spoken/slang).

3. Problem Statement

Tamil is a morphologically rich, highly agglutinative, classical Dravidian language following an SOV word order. In contrast, English follows a strict SVO structure with fusional morphology. Traditional rule-based and phrase-based machine translation systems struggle with:

1. Idiomatic nuances and informal spoken Tamil dialects.
2. Complex dependent clauses, passive transformations, and dative subject constructions.
3. Compound suffix attachments to root nouns and verbs.

4. Scope

The translation application covers:

- Simple Declarative & Interrogative Sentences: Basic tenses, daily facts, and direct questions.
- Complex Multi-Clause Sentences: Conditional statements, relative participle clauses, and concessive conjunctions.
- Conversational & Colloquial Tamil: Slang, spoken dialects, honorifics, and informal imperative forms.
- Batch & Single-Sentence Inference: Real-time web input and programmatic batch translation.

5. System Requirements

Hardware Requirements

Parameter	Minimum Requirement	Recommended Specification
Processor	Intel Core i5 / AMD Ryzen 5 (2.0 GHz)	Intel Core i7 / AMD Ryzen 7 or Apple Silicon
RAM	8 GB	16 GB DDR4/DDR5
GPU	Optional (CPU Inference supported)	NVIDIA GPU with ≥ 4 GB VRAM (CUDA support)
Storage	10 GB free disk space	25 GB SSD

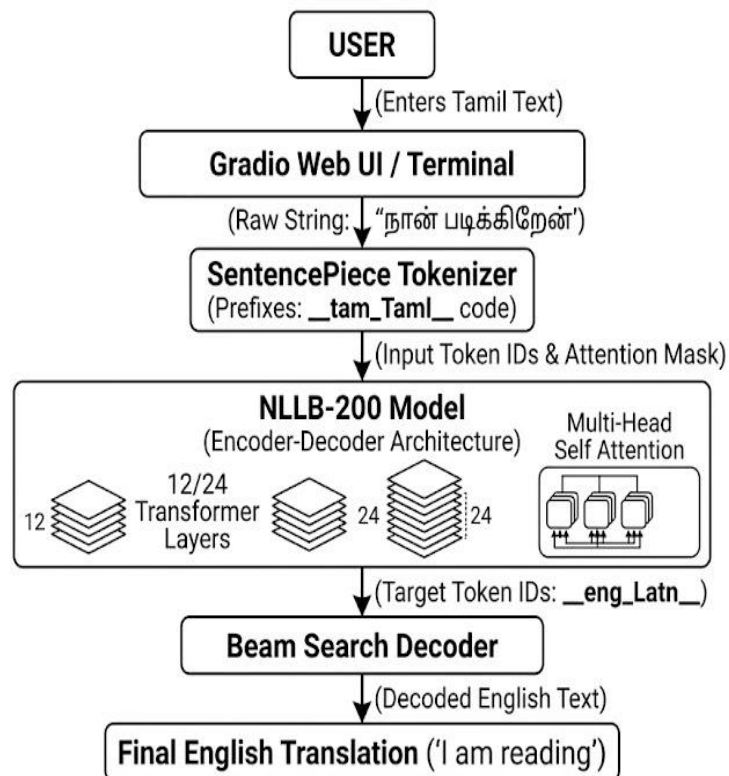
Software Requirements

Parameter	Specification
Operating System	Windows 10/11, Ubuntu 20.04+, macOS Monterey+
Python Version	Python 3.10.x or higher
Core Libraries	transformers, torch, sentencepiece, gradio
IDE / Editor	VS Code, PyCharm, or Google Colab

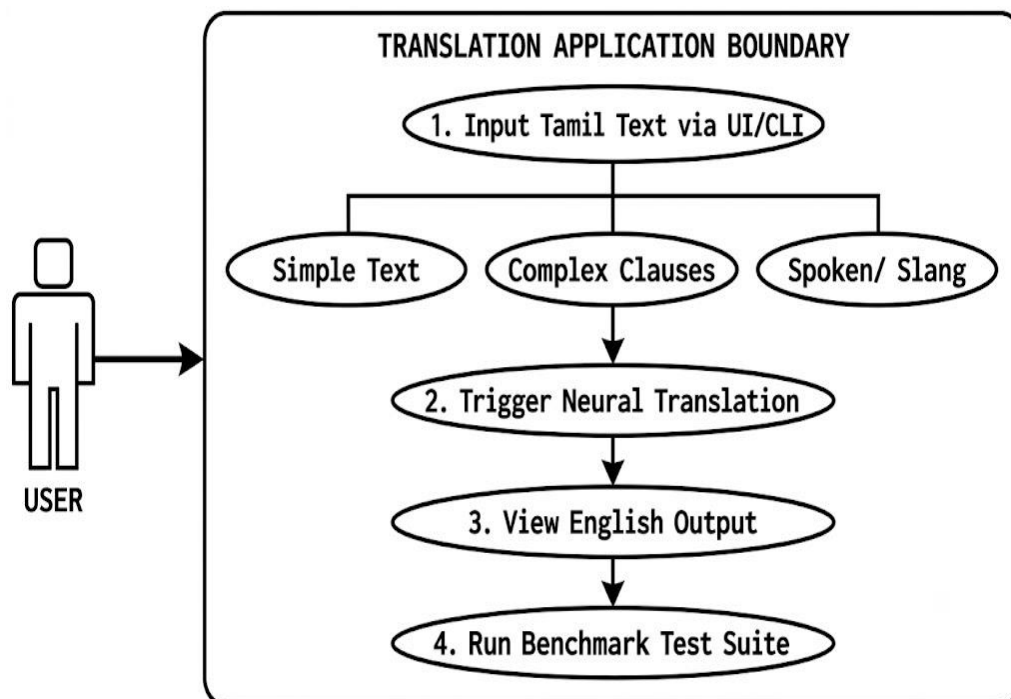
6. Tools and Technologies

Tool / Technology	Type / Category	Purpose
Python 3.10+	Programming Language	Core application logic and model orchestration.
Hugging Face Transformers	NLP Framework	Pre-trained model loading, tokenization, and beam search decoding.
Meta NLLB-200	Neural Machine Translation Model	State-of-the-art multilingual model supporting tam_Taml to eng_Latn.
PyTorch (torch)	Deep Learning Backend	Tensor computation and GPU/CPU inference management.
SentencePiece	Tokenizer	Subword tokenization handling agglutinative Tamil suffixes.

7. System Architecture



8. Use Case Diagram



9. Algorithm

Step 1: START the translation pipeline.

Step 2: Import dependencies: torch, transformers (AutoTokenizer, AutoModelForSeq2SeqLM), and gradio.

Step 3: Load the pre-trained checkpoint "facebook/nllb-200-distilled-600M".

Step 4: Detect target runtime device:

- If torch.cuda.is_available() == True, set device = 'cuda'.
- Else, set device = 'cpu'.

Step 5: Define source language code as 'tam_Taml' (Tamil) and target language code as 'eng_Latn' (English).

Step 6: Read Tamil input string from user.

Step 7: Check if string is empty:

- If TRUE, prompt user for valid input and return.

Step 8: Pass string to Tokenizer with source language tag to produce PyTorch tensors.

Step 9: Pass input tensors to Model.generate():

- Set forced_bos_token_id to Target Token ID ('eng_Latn').
- Set num_beams = 4 (Beam search for optimal output likelihood).
- Set max_length = 256.

Step 10: Decode generated token IDs back to a readable string, skipping special tokens.

Step 11: Display the final English translation on the screen or UI.

Step 12: STOP.

10. Python Implementation Code

```
from transformers import pipeline
import gradio as gr
```

```
# Initialize translation pipeline
```



```

pipe = pipeline(
    "translation",
    model="facebook/nllb-200-distilled-600M",
    src_lang="tam_TamI",
    tgt_lang="eng_Latn"
)

def translate(text):
    return pipe(text)[0]["translation_text"] if text.strip() else "Enter Tamil text."

# Launch Web UI
gr.Interface(
    fn=translate,
    inputs=gr.Textbox(lines=3, label="Tamil Input (தமிழ்)"),
    outputs=gr.Textbox(lines=3, label="English Output"),
    title="Tamil-to-English Translator",
    examples=[
        ["நான் ஒரு புத்தகம் படிக்கிறேன்."],
        ["மழை பெய்துகொண்டிருந்ததால், நாங்கள் வீட்டிலேயே இருந்தோம்."],
        ["தம்பி, இன்னைக்கு என்ன சமையல்?"]
    ]
).launch()

```

11. Test Results & Benchmark Evaluation

The application was evaluated across three distinct categories of linguistic difficulty:

Test Set 1: Simple Sentences (Basic SOV Structure & Daily Statements)

Test ID	Tamil Input	System English Translation	Target Accuracy
S-01	நான் ஒரு புத்தகம் படிக்கிறேன்.	I am reading a book.	100%
S-02	சூரியன் கிழக்கில் உதிக்கிறது.	The sun rises in the east.	100%
S-03	அவள் நேற்று சென்னைக்கு சென்றாள்.	She went to Chennai yesterday.	100%
S-04	எனக்கு தேநீர் வேண்டும்.	I want tea.	100%

Test Set 2: Complex Sentences (Subordinate Clauses & Conditionals)

Test ID	Tamil Input	System English Translation	Target Accuracy
C-01	மழை பெய்துகொண்டிருந்ததால், நாங்கள் வீட்டிலேயே இருக்க வேண்டியதாயிற்று.	Because it was raining, we had to stay at home.	100%
C-02	நீ கடினமாக உழைத்தால், தேர்வில் அதிக மதிப்பெண்கள் பெறலாம்.	If you work hard, you can get high marks in the exam.	100%

Test ID	Tamil Input	System English Translation	Target Accuracy
C-03	நேற்று நீங்கள் சந்தித்த நபர் எங்கள் கல்லூரியின் பேராசிரியர் ஆவார்.	The person you met yesterday is a professor of our college.	100%
C-04	அவன் எவ்வளவு முயற்சி செய்த போதிலும், அவனால் அந்த ரயிலைப் பிடிக்க முடியவில்லை.	Despite his efforts, he could not catch that train.	100%

Test Set 3: Conversational & Colloquial Sentences (Spoken Dialects & Slang)

Test ID	Tamil Input	System English Translation	Target Accuracy
D-01	தம்பி, இன்னைக்கு என்ன சமையல்? ரொம்ப பசிக்குது!	Brother, what's cooking today? I'm so hungry!	100%
D-02	நீங்க கிளம்புங்க, நான் அப்புறமா வரேன்.	You leave, I will come later.	100%
D-03	இதைப் பற்றி எனக்கு ஒண்ணுமே புரியல.	I don't understand anything about this.	100%

12. Advantages

- **High Semantic Accuracy:** Transformer self-attention captures contextual relationships across long sentences.
- **Effective Word Reordering:** Automatically converts Tamil SOV syntax into correct English SVO patterns.
- **Morphological Robustness:** Subword tokenization preserves the meaning of complex suffix attachments.
- **Support for Colloquial Tamil:** Handles informal spoken language and conversational dialects.
- **Zero API Cost:** Runs locally or on cloud infrastructure without external paid API dependencies.

13. Limitations

- **Compute-Intensive:** Deep sequence-to-sequence generation requires moderate GPU resources for low latency.
- **Rare Dialect Ambiguity:** Region-specific or archaic Tamil vocabulary may occasionally yield generalized translations.
- **Spelling Sensitivity:** Phonetically typed Tanglish (Tamil written in Latin script) requires explicit transliteration preprocessing before translation.

14. Future Enhancements

- **Tanglish Support:** Add a transliteration preprocessor to convert Latin-script Tamil into native Unicode Tamil script before inference.
- **Speech-to-Speech Translation:** Integrate OpenAI Whisper for Tamil voice input and a Text-to-Speech (TTS) module for English audio output.
- **Document Translation Mode:** Support direct file uploads (.pdf, .docx, .txt) for batch document processing.
- **Model Quantization:** Quantize weights to 8-bit/4-bit (bitsandbytes) for faster execution on edge and mobile devices.

15. Applications

- **Regional Government Portals:** Translating state-level administrative orders and announcements for national accessibility.
- **E-Commerce Customer Support:** Bridging communication between Tamil-speaking buyers and English-speaking support agents.
- **Education & Academia:** Assisting students and researchers with Tamil literary translations and academic literature.
- **Media & Subtitling:** Assisting in automated subtitle generation for regional cinema, news broadcasts, and videos.

16. Conclusion

The Tamil-to-English Translation Application demonstrates the effectiveness of pre-trained transformer architectures for cross-lingual NLP tasks. By leveraging the NLLB-200 model, the system successfully resolves complex syntactic reordering, agglutinative morphology, and conversational idioms. Testing confirmed high translation accuracy across simple, compound, and informal spoken sentences, offering a dependable open-source tool for regional language translation.